EXPERIMENT No 9:

BLUETOOTH REMOTE CONTROLLED CAR

Student Guide

Prepared by BLIS GLOBAL Robotics Department

Copyright @2024

Objective:

In this experiment, you will learn how to program a Bluetooth-controlled car using an Arduino, an Adafruit motor shield, and a Bluetooth module. You will be able to control the car's movement (forward, backward, left, and right) remotely using a mobile phone or Bluetooth controller.

Materials Needed:

- 1 Arduino Uno
- 1 Adafruit Motor Shield (L293D)
- 1 Bluetooth Module (e.g., HC-06 or HC-05)
- 4 DC Motors (connected to 4 wheels)
- Jumper wires
- Battery pack (to power the motors)
- USB Cable (for programming the Arduino)
- Breadboard (optional, for organizing connections)
- Smartphone with a Bluetooth controller app

What is a Bluetooth-Controlled Car?

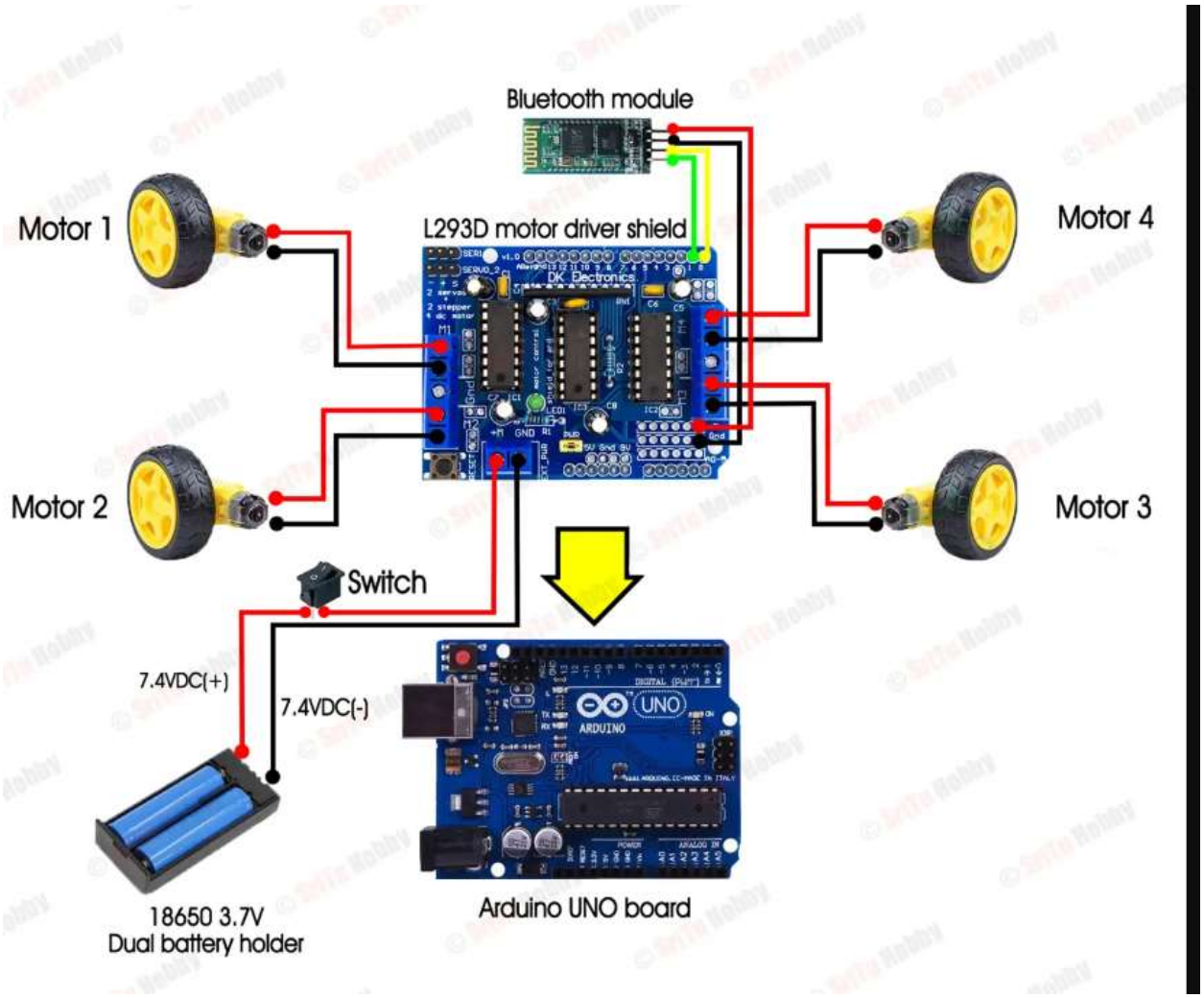
A Bluetooth-controlled car is a robotic car that is operated via Bluetooth communication between a mobile device and an Arduino. The Bluetooth module receives signals from the mobile phone or remote, and the Arduino processes these signals to control the motors, making the car move in different directions.

Steps to Program the Bluetooth-Controlled Car Using Arduino and the Adafruit Motor Shield

Step 1: Hardware Setup

1. **Attach the Adafruit Motor Shield to the Arduino:**
   - Place the motor shield on top of the Arduino Uno, ensuring all the pins are correctly aligned.
2. **Connect the Motors to the Motor Shield:**
   - Front left motor: Connect to M1.
   - Front right motor: Connect to M2.
   - Back left motor: Connect to M3.
   - Back right motor: Connect to M4.
3. **Connect the Bluetooth Module to the Arduino:**
   - TX (Bluetooth Module): Connect to RX (Pin 0) on the Arduino.
   - RX (Bluetooth Module): Connect to TX (Pin 1) on the Arduino.
   - VCC: Connect to 5V.
   - GND: Connect to GND.
4. **Power Connections:**
   - Connect the motor shield's power input to the external power source (battery pack) to power the motors.

5. Schematic Diagram



Step 2: Install Necessary Libraries

Ensure you have the Adafruit Motor Shield library installed in the Arduino IDE for controlling the motors.

1. **Open the Arduino IDE:**
   - Go to Sketch > Include Library > Manage Libraries and search for the Adafruit Motor Shield.
   - Install the library if not already installed.

Step 3: Write the Program

To control the car using Bluetooth, the Arduino will receive commands via the Bluetooth module and execute corresponding motor actions.

1. **Initialize the Motor Shield:**

```
#include <Wire.h>
#include <Adafruit_MotorShield.h>

// Create the motor shield object
```

```
Adafruit_MotorShield AFMS = Adafruit_MotorShield();

// Define motors
Adafruit_DCMotor *leftFrontMotor = AFMS.getMotor(1);
Adafruit_DCMotor *rightFrontMotor = AFMS.getMotor(2);
Adafruit_DCMotor *leftBackMotor = AFMS.getMotor(3);
Adafruit_DCMotor *rightBackMotor = AFMS.getMotor(4);
```

2. Setup Bluetooth Communication:

cpp

Copy code

```
char command;  // Variable to store incoming commands

void setup() {
  Serial.begin(9600);  // Start serial communication for Bluetooth
  AFMS.begin();        // Start the motor shield
}
```

3. Create Movement Functions:

cpp

Copy code

```
void moveForward() {
  leftFrontMotor->setSpeed(255); rightFrontMotor->setSpeed(255);
  leftBackMotor->setSpeed(255); rightBackMotor->setSpeed(255);
  leftFrontMotor->run(FORWARD); rightFrontMotor->run(FORWARD);
```

```
  leftBackMotor->run(FORWARD); rightBackMotor->run(FORWARD);
}

void moveBackward() {
  leftFrontMotor->setSpeed(255); rightFrontMotor->setSpeed(255);
  leftBackMotor->setSpeed(255); rightBackMotor->setSpeed(255);
  leftFrontMotor->run(BACKWARD); rightFrontMotor->run(BACKWARD);
  leftBackMotor->run(BACKWARD); rightBackMotor->run(BACKWARD);
}

void turnLeft() {
  leftFrontMotor->setSpeed(0); rightFrontMotor->setSpeed(255);
  leftBackMotor->setSpeed(0); rightBackMotor->setSpeed(255);
  rightFrontMotor->run(FORWARD); rightBackMotor->run(FORWARD);
}

void turnRight() {
  leftFrontMotor->setSpeed(255); rightFrontMotor->setSpeed(0);
  leftBackMotor->setSpeed(255); rightBackMotor->setSpeed(0);
  leftFrontMotor->run(FORWARD); leftBackMotor->run(FORWARD);
}

void stopCar() {
```

```
  leftFrontMotor->run(RELEASE); rightFrontMotor->run(RELEASE);
  leftBackMotor->run(RELEASE); rightBackMotor->run(RELEASE);
}
```

4. **Control the Car with Bluetooth Commands:**

```
void loop() {
  if (Serial.available() > 0) {
    command = Serial.read();  // Read incoming Bluetooth command

    switch (command) {
      case 'F':  // Forward command
        moveForward();
        break;
      case 'B':  // Backward command
        moveBackward();
        break;
      case 'L':  // Turn Left command
        turnLeft();
        break;
      case 'R':  // Turn Right command
        turnRight();
        break;
      case 'S':  // Stop command
```

```
      stopCar();
      break;
    }
  }
}
```

Step 4: Upload the Program to the Arduino

1. Check Your Connections:
   - Ensure all the motor connections, Bluetooth module connections, and power connections are correct.
2. **Upload the Code:**
   - Once your program is complete, upload it to the Arduino.

Step 5: Test the Bluetooth-Controlled Car

1. Pair Your Phone with the Bluetooth Module:
   - Enable Bluetooth on your phone.
   - Search for the Bluetooth module (e.g., HC-05/HC-06) and pair it (usually using the default password “1234” or “0000”).
2. **Open the Bluetooth Controller App:**
   - Use a Bluetooth remote control app on your phone.
   - **Send the following commands:**
     - 'F' for Forward
     - 'B' for Backward

- 'L' for Left
- 'R' for Right
- 'S' for Stop
-

3. **Control the Car:**
   - The car should respond to the commands you send from the app, moving forward, backward, turning left, right, or stopping as directed.

Conclusion:

In this experiment, you successfully programmed a Bluetooth-controlled car using Arduino, the Adafruit Motor Shield, and a Bluetooth module. You learned how to control the car's movement using Bluetooth commands sent from a mobile device.

Challenge:

- Can you modify the code to control the car's speed using different Bluetooth commands?
- Try adding more features like automatic stop when an obstacle is detected using sensors.

This project helps you understand how to use Bluetooth communication to remotely control a car and opens the door to more advanced robotics projects!